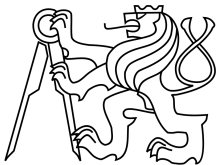


Formal Methods and Specification (SS 2025/26)

Modeling of Data Structures 2 and Object Oriented Programming

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Global Plan

software systems

...

modularization

functions, procedures

control structures

basic paths:

- assume** $x \geq 5$
- $y \leftarrow x$
- $z \leftarrow x + y$
- @ $z \geq 0$

logic, proofs

Abstract Data Types

A data type with certain operations that is **independent** of any **implementation**.

Example:

Signature:

init: $\mathcal{N} \rightarrow \text{counter}$

val: $\text{counter} \rightarrow \mathcal{N}$

dec: $\text{counter} \rightarrow \text{counter}$

Axioms:

$$\forall n \in \mathcal{N} . \text{val}(\text{init}(n)) = n$$

$$\forall c \in \text{counter}, n \in \mathcal{N} . \left[\begin{array}{l} \text{val}(c) = 0 \Rightarrow \text{val}(\text{dec}(c)) = 0 \wedge \\ \text{val}(c) \neq 0 \Rightarrow \text{val}(\text{dec}(c)) = \text{val}(c) - 1 \end{array} \right]$$

Description of abstract data types: signature + **axioms**

This is nothing else than a logical **theory**! C++ header files, Java interfaces

Object Oriented Programming

≈ abstract data types + inheritance + dynamic binding + strange notation

Example: notation for signature

init: $\mathcal{N} \rightarrow \text{counter}$
val: counter $\rightarrow \mathcal{N}$
dec: counter $\rightarrow \text{counter}$

```
class counter  
constructor init(n)  
method val():  $\mathcal{N}$   
method dec()
```

Usage example:

$i \leftarrow \text{val}(c)$

$i \leftarrow c.\text{val}()$

How to write axioms?

Axioms in OO Notation

Classical Notation:

$$\forall n \in \mathcal{N} . \text{val}(\text{init}(n)) = n$$

$$\forall c \in \text{counter}, n \in \mathcal{N} . \left[\begin{array}{l} \text{val}(c) = 0 \Rightarrow \text{val}(\text{dec}(c)) = 0 \wedge \\ \text{val}(c) \neq 0 \Rightarrow \text{val}(\text{dec}(c)) = \text{val}(c) - 1 \end{array} \right]$$

OO Notation:

$$\forall n \in \mathcal{N} . \text{init}(n).\text{val}() = n$$

$$\forall c . \left[\begin{array}{l} c.\text{val}() = 0 \Rightarrow c.\text{dec}().\text{val}() = 0 \wedge \\ c.\text{val}() \neq 0 \Rightarrow c.\text{dec}().\text{val}() = c.\text{val}() - 1 \end{array} \right]$$

Writing Axioms as Assertions

$$\forall n \in \mathcal{N} . \text{init}(n).\text{val}() = n$$

$$\forall c . \left[\begin{array}{l} c.\text{val}() = 0 \Rightarrow c.\text{dec}().\text{val}() = 0 \wedge \\ c.\text{val}() \neq 0 \Rightarrow c.\text{dec}().\text{val}() = c.\text{val}() - 1 \end{array} \right]$$

class counter

constructor init(n)

assume $n \in \mathcal{N}$

 ...

 @ $\text{val}() = n$

method $\text{val}(): \mathcal{N}$

method dec()

$v \leftarrow \text{val}()$

 ...

 @ $[v = 0 \Rightarrow \text{val}() = 0] \wedge [v \neq 0 \Rightarrow \text{val}() = v - 1]$

Verification Conditions in Natural Notation

Assumptions from specification:

$$\forall n \in \mathcal{N} . \text{val}(\text{init}(n)) = n$$

$$\forall c \in \text{counter} . \left[\begin{array}{l} \text{val}(c) = 0 \Rightarrow \text{val}(\text{dec}(c)) = 0 \wedge \\ \text{val}(c) \neq 0 \Rightarrow \text{val}(\text{dec}(c)) = \text{val}(c) - 1 \end{array} \right]$$

$c \leftarrow \text{init}(2)$	// $c \leftarrow \text{init}(2)$
@ $c.\text{val}() = 2$	// @ $\text{val}(c) = 2$
$v \leftarrow c.\text{val}()$	// $v \leftarrow \text{val}(c)$
$c.\text{dec}()$	// $c \leftarrow \text{dec}(c)$
@ $c.\text{val}() = v - 1$	// @ $\text{val}(c) = v - 1$

Verification conditions (without assumptions from above):

- ▶ $c = \text{init}(2) \Rightarrow \text{val}(c) = 2$
- ▶ $[\text{val}(c) = 2 \wedge v = \text{val}(c) \wedge c_1 = \text{dec}(c)] \Rightarrow \text{val}(c_1) = v - 1$

Verification Conditions in Strange Notation

Assumptions from the specification (dots in the formulas are only notation)

$$\forall n \in \mathcal{N} . \text{init}(n).\text{val}() = n$$

$$\forall c . \left[\begin{array}{l} c.\text{val}() = 0 \Rightarrow c.\text{dec}().\text{val}() = 0 \wedge \\ c.\text{val}() \neq 0 \Rightarrow c.\text{dec}().\text{val}() = c.\text{val}() - 1 \end{array} \right]$$

$c \leftarrow \text{init}(2)$

$@ \ c.\text{val}() = 2$

$v \leftarrow c.\text{val}()$

$c.\text{dec}()$

$@ \ c.\text{val}() = v - 1$

// $@ \ \text{val}(c) = 2$

// $v \leftarrow \text{val}(c)$

// $c \leftarrow \text{dec}(c)$

// $@ \ \text{val}(c) = v - 1$

Verification conditions:

► $c = \text{init}(2) \Rightarrow c.\text{val}() = 2$

► $[c.\text{val}() = 2 \wedge v = c.\text{val}() \wedge c_1 = c.\text{dec}()] \Rightarrow c_1.\text{val}() = v - 1$

Internal Consistency of Data Types

Example:

The internal variable x should always fulfill $x \geq 0$.

This would amount to an assertion at
the beginning and end of **every** method.

Some object oriented languages have explicit support:

Eiffel: **invariant** $x \geq 0$,
is checked at the beginning and end of every method

Everything we have seen up to now,
can also be expressed (maybe even better) without OO constructions.

see also: design by contract
(Eiffel, Microsoft Code Contracts, Java Modeling Language, ...)

see also: wikipedia

Abstract Data Types

Rest of lecture:

- ▶ example of an ADT as a logical theory
- ▶ subtyping
- ▶ building new ADTs based on old ones

Example: Axiomatic Specification of FIFO Queue

Signature:

$\text{emptyq}: \rightarrow \mathcal{Q}$

$\text{enqueue}: \mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

$\text{get}: \mathcal{Q} \rightarrow \mathcal{N}$

$\text{dequeue}: \mathcal{Q} \rightarrow \mathcal{Q}$

Axioms:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

$$\forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) = \text{emptyq}()$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow$$

$$\text{dequeue}(\text{enqueue}(x, q)) = \text{enqueue}(x, \text{dequeue}(q))$$

This leaves the behavior of dequeue/get on an empty queue open.

Application

Axioms:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

$$\forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) = \text{emptyq}()$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow$$

$$\text{dequeue}(\text{enqueue}(x, q)) = \text{enqueue}(x, \text{dequeue}(q))$$

Expression simplification:

$$\text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq}())))) = \text{get}(\text{enqueue}(3, \text{emptyq}())) = 3$$

and further **proofs** of behavior of data type, **independent of implementation**

implementation: `queue.hs`

Implementation: Terminology

depends on concrete programming language

C++/Java:

- ▶ signature: interface/abstract class
- ▶ ADT: behavior only expressible by comments
- ▶ implementation of ADT: concrete class
- ▶ elements of concrete type: objects

Now: implementation by reduction to known data types

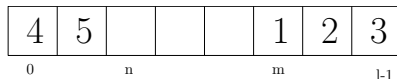
- ▶ allows for correctness check wrt. axioms
- ▶ independent of concrete programming language

Implementation: Representation

Intuition: Represent, for example,

`enqueue(1, enqueue(2, enqueue(3, enqueue(4, enqueue(5, emptyq())))))`

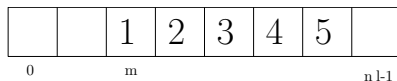
by



l : global constant

$\mathcal{Q} := \mathcal{A} \times \mathcal{N} \times \mathcal{N}$

problem?



equality? unique representation?

Example: Type-Specific Equality

Signature:

$\text{emptyq}: \rightarrow \mathcal{Q}$

$\text{enqueue}: \mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

$\text{get}: \mathcal{Q} \rightarrow \mathcal{N}$

$\text{dequeue}: \mathcal{Q} \rightarrow \mathcal{Q}$

$=_{\mathcal{Q}}: \mathcal{Q} \times \mathcal{Q}$

Axioms:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq_{\mathcal{Q}} \text{emptyq}()$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq_{\mathcal{Q}} \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

$$\forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) =_{\mathcal{Q}} \text{emptyq}()$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq_{\mathcal{Q}} \text{emptyq}() \Rightarrow$$

$$\text{dequeue}(\text{enqueue}(x, q)) =_{\mathcal{Q}} \text{enqueue}(x, \text{dequeue}(q))$$

+ reflexivity, symmetry, transitivity, congruence of $=_{\mathcal{Q}}$

Implementation of Functions and Predicates

function emptyq : $\rightarrow \mathcal{Q}$

Output: $(a, 0, 0)$

function enqueue : $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

Input: $(x, (a, m, n))$

Output: $(write(a, (m - 1) \bmod l, x), (m - 1) \bmod l, n)$

function get : $\mathcal{Q} \rightarrow \mathcal{N}$

Input: (a, m, n)

Output: $a[(n - 1) \bmod l]$

function dequeue : $\mathcal{Q} \rightarrow \mathcal{Q}$

Input: (a, m, n)

Output: $(a, m, (n - 1) \bmod l)$

predicate equal : $\mathcal{Q} \times \mathcal{Q}$

Input: $(a_1, m_1, n_1), (a_2, m_2, n_2)$

Output: $(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$
 $\forall i. 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$

Correctness of Implementation

axioms: basis for constructing test cases!

interface	function interface formulas ADT (signature+axioms)
implementation	concrete type and functions

Correctness of Implementation

function emptyq : $\rightarrow \mathcal{Q}$

Output: $(a, 0, 0)$

$$\text{emptyq}() = (a, 0, 0)$$

function enqueue : $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

Input: $(x, (a, m, n))$

Output: $(\text{write}(a, (m-1) \bmod l, x), (m-1) \bmod l, n)$

$$\forall x \in \mathcal{N}, (a, m, n) \in \mathcal{Q} .$$

$$\text{enqueue}(x, (a, m, n)) = (\text{write}(a, (m-1) \bmod l, x), (m-1) \bmod l, n)$$

function get : $\mathcal{Q} \rightarrow \mathcal{N}$

Input: (a, m, n)

Output: $a[(n-1) \bmod l]$

$$\forall (a, m, n) \in \mathcal{Q} . \text{get}((a, m, n)) = a[(n-1) \bmod l]$$

function dequeue : $\mathcal{Q} \rightarrow \mathcal{Q}$

Input: (a, m, n)

Output: $(a, m, (n-1) \bmod l)$

$$\forall (a, m, n) \in \mathcal{Q} . \text{dequeue}(a, m, n) = (a, m, (n-1) \bmod l)$$

Correctness of Implementation

predicate `equal` : $\mathcal{Q} \times \mathcal{Q}$

Input: $(a_1, m_1, n_1), (a_2, m_2, n_2)$

Output: $(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$
 $\forall i . 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$

$$\forall (a_1, m_1, n_1), (a_2, m_2, n_2) \in \mathcal{Q} \times \mathcal{Q} . (a_1, m_1, n_1) =_{\mathcal{Q}} (a_2, m_2, n_2) \Leftrightarrow$$

$$(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$$

$$\forall i . 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$$

Correctness of Implementation

Assumptions:

$$\text{emptyq}() = (a, 0, 0)$$

$$\forall x \in \mathcal{N}, (a, m, n) \in \mathcal{Q} . \text{enqueue}(x, (a, m, n)) = \\ (\text{write}(a, (m-1) \bmod l, x), (m-1) \bmod l, n)$$

$$\forall (a, m, n) \in \mathcal{Q} . \text{get}((a, m, n)) = a[(n-1) \bmod l]$$

$$\forall (a, m, n) \in \mathcal{Q} . \text{dequeue}(a, m, n) = (a, m, (n-1) \bmod l)$$

$$\forall (a_1, m_1, n_1), (a_2, m_2, n_2) \in \mathcal{Q} \times \mathcal{Q} . (a_1, m_1, n_1) =_{\mathcal{Q}} (a_2, m_2, n_2) \Leftrightarrow \\ (n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge \\ \forall i . 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$$

Prove:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq_{\mathcal{Q}} \text{emptyq}()$$

$$(\text{write}(a, (m-1) \bmod l, x), (m-1) \bmod l, n) \neq_{\mathcal{Q}} (a, 0, 0) ???$$

overflow possible!!!

Correctness of Implementation

Assumptions: $\text{emptyq}() = (a, 0, 0)$

$$\forall x \in \mathcal{N}, (a, m, n) \in \mathcal{Q} . \text{enqueue}(x, (a, m, n)) = \\ (\text{write}(a, (m - 1) \bmod l, x), (m - 1) \bmod l, n)$$

$$\forall (a, m, n) \in \mathcal{Q} . \text{get}((a, m, n)) = a[(n - 1) \bmod l]$$

$$\forall (a, m, n) \in \mathcal{Q} . \text{dequeue}(a, m, n) = (a, m, (n - 1) \bmod l)$$

$$\forall (a_1, m_1, n_1), (a_2, m_2, n_2) \in \mathcal{Q} \times \mathcal{Q} . (a_1, m_1, n_1) =_{\mathcal{Q}} (a_2, m_2, n_2) \Leftrightarrow$$

$$(n_1 - m_1) \bmod l = (n_2 - m_2) \bmod l \wedge$$

$$\forall i . 0 \leq i < (n_1 - m_1) \bmod l \Rightarrow a_1[(m_1 + i) \bmod l] = a_2[(m_2 + i) \bmod l]$$

Prove:

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, (a, 0, 0))) = x$$

$$\forall x \in \mathcal{N} . \text{get}(\text{write}(a, -1 \bmod l, x), -1 \bmod l, 0) = x$$

$$\forall x \in \mathcal{N} . \text{write}(a, -1 \bmod l, x)[-1 \bmod l] = x$$

Correctness of Implementation

interface	function interface formulas ADT (signature+axioms)
implementation	concrete type and functions

If

- ▶ the function interface formulas imply the axioms
- ▶ the function implementations satisfy their interfaces

then the implementation of the ADT is correct.

Terminology in Logic

interface	function interface formulas	logical theory
implementation	<i>structure</i>	

Implementing Abstract Data Types

Let Σ be a signature over types $T_1, \dots, T_n$.

A Σ -*structure* S is a tuple $(\Omega_1, \dots, \Omega_n, \mathcal{I})$ where

- ▶ $\Omega_1, \dots, \Omega_n$ are sets (the *carrier sets* of S)
- ▶ $\mathcal{I}$ assigns to
 - ▶ every function symbol $f : T_{i_1} \times \dots \times T_{i_a} \rightarrow T_j$ from Σ
a function $f : \Omega_{i_1} \times \dots \times \Omega_{i_a} \rightarrow \Omega_j$
 - ▶ every predicate symbol $p : T_{i_1} \times \dots \times T_{i_a}$ from Σ
a relation $r \subseteq \Omega_{i_1} \times \dots \times \Omega_{i_a}$

$$\text{queue: } \left(\mathcal{A} \times \mathcal{N} \times \mathcal{N}, \left\{ \begin{array}{l} \text{emptyq} \mapsto \text{emptyq}, \\ \text{enqueue} \mapsto \text{enqueue}, \\ \text{get} \mapsto \text{get}, \\ \text{dequeue} \mapsto \text{dequeue}, =_Q \mapsto \text{equal} \end{array} \right\} \right)$$

Correctness of Implementation in Math and Logic

A structure may or may not satisfy certain properties

Examples:

- ▶ $\mathbb{Z}_2$ satisfies $1 + 1 = 0$
- ▶ $\mathbb{Z}$ does not satisfy $1 + 1 = 0$
- ▶ the rational numbers with addition satisfy the axioms of group theory

Logical terminology: a structure is a *model* of a theory

Attention: the term “model” usually means something different!

(Rough) Translation Table

software engineering	logic
interface (Java)	signature
(specification of) ADT	logical theory (signature+axioms)
concrete class	structure
object	element of carrier of structure
correct implementation	model

Extensions of Data Types

Sometimes we want to **add function/predicate symbols** to an existing specification, e.g.:

- ▶ add a length function to the queue spec
- ▶ add an error constant to the queue spec

Sometimes we want to **add properties** to an existing specification, e.g.:

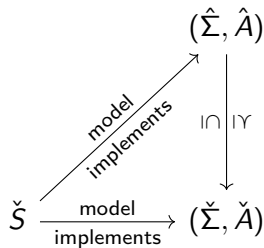
- ▶ add the axiom $\text{dequeue}(\text{emptyq}()) = \text{emptyq}()$
- ▶ restrict groups to commutative groups

Given two logical theories $(\check{\Sigma}, \check{A})$ and $(\hat{\Sigma}, \hat{A})$,
 $(\check{\Sigma}, \check{A})$ is an **extension** of $(\hat{\Sigma}, \hat{A})$ iff $\check{\Sigma} \supseteq \hat{\Sigma}$ and $\check{A} \supseteq \hat{A}$

Notation: $(\check{\Sigma}, \check{A}) \preceq (\hat{\Sigma}, \hat{A})$



Extensions of Data Types



For two logical theories $(\check{\Sigma}, \check{A}) \preceq (\hat{\Sigma}, \hat{A})$
for every model $\check{S}$ of $(\check{\Sigma}, \check{A})$,
the restriction of $\check{S}$ to $\hat{\Sigma}$ is a model of $(\hat{\Sigma}, \hat{A})$

Intuition:

every implementation of $(\check{\Sigma}, \check{A})$ is also an implementation of $(\hat{\Sigma}, \hat{A})$

something similar in object-oriented programming?

Subtyping vs. Theory Extension

Subtypes should ensure the **same properties** as the supertype when used in the same way

Liskov substitution principle:

“Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should be true for objects y of type S where S is a subtype of T ”.
[Liskov and Wing, 1994]



In **OO** programming languages only partially ensured, **syntactically** (e.g., rules of co/contravariance)

Theory extensions guarantee this, but do not take into account usual phenomena of OO languages

(Rough) Translation Table

software engineering	logic
interface (Java)	signature
(specification of) ADT	logical theory (signature+axioms)
concrete class	structure
object	element of carrier of structure
correct implementation	model
subtype	theory extension

Definition of Abstract Data Types by Extensions

Up to now ADT defined purely intensionally (based on their properties)

But we already know many data types (i.e., logical theories)

Why start **from scratch**?

Build new data type based on **existing** ones?

Example: We defined strings as pairs of

- ▶ an array of characters, and
- ▶ the length of the string.

Lecture notes:

Example “String Matching” in chapter “Program Specification”

Example: FIFO Queue by Extension

We define queues $\mathcal{Q}$ by **extending lists** $\mathcal{L}$

Signature: $\mathcal{L}+$

emptyq: $\rightarrow \mathcal{Q}$

enqueue: $\mathcal{N} \times \mathcal{Q} \rightarrow \mathcal{Q}$

get: $\mathcal{Q} \rightarrow \mathcal{N}$

dequeue: $\mathcal{Q} \rightarrow \mathcal{Q}$

Axioms: $\mathcal{L}+$

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, \forall q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

where last, dropLast are defined in $\mathcal{L}$, e.g. as follows:

$$\text{last}(\langle x \rangle) = x, l \neq \langle \rangle \Rightarrow \text{last}(\text{cons}(x, l)) = \text{last}(l)$$

$$\text{dropLast}(\langle x \rangle) = \langle \rangle, l \neq \langle \rangle \Rightarrow \text{dropLast}(\text{cons}(x, l)) = \text{cons}(x, \text{dropLast}(l))$$

Application: Expression Simplification

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, \forall q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

Expression simplification:

$$\text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq()}))) =$$

$$\text{get}(\text{enqueue}(7, \text{enqueue}(3, \langle \rangle))) =$$

$$\text{get}(\text{enqueue}(7, \langle 3 \rangle)) =$$

$$\text{get}(\langle 7, 3 \rangle) = 3$$

Comparison

Intensional:

$$\begin{aligned} \text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq()}))) &= \\ \text{get}(\text{enqueue}(3, \text{emptyq()})) &= 3 \end{aligned}$$

Extension Based:

$$\begin{aligned} \text{get}(\text{enqueue}(7, \text{enqueue}(3, \text{emptyq()}))) &= \\ \text{get}(\text{enqueue}(7, \text{enqueue}(3, \langle \rangle))) &= \\ \text{get}(\text{enqueue}(7, \langle 3 \rangle)) &= \\ \text{get}(\langle 7, 3 \rangle) &= 3 \end{aligned}$$

Intensional vs. Extending Definition of ADT

Intensional axioms:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

$$\forall x \in \mathcal{N} . \text{dequeue}(\text{enqueue}(x, \text{emptyq}())) = \text{emptyq}()$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow$$

$$\text{dequeue}(\text{enqueue}(x, q)) = \text{enqueue}(x, \text{dequeue}(q))$$

Extending axioms:

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

Does the extending definition fulfill the intensional axioms?

Correctness Proof

Axiom:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) \neq \text{emptyq}()$$

List-based spec:

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

Proof:

$$\text{enqueue}(x, q) = \text{cons}(x, q) \neq \langle \rangle = \text{emptyq}()$$

Correctness Proof

Axiom:

$$\forall x \in \mathcal{N} . \text{get}(\text{enqueue}(x, \text{emptyq}())) = x$$

List-based spec:

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

Proof:

$$\text{get}(\text{enqueue}(x, \text{emptyq}())) = \text{get}(\text{enqueue}(x, \langle \rangle)) = \text{get}(\langle x \rangle) = \text{last}(\langle x \rangle) = x$$

Correctness Proof

Axiom:

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . q \neq \text{emptyq}() \Rightarrow \text{get}(\text{enqueue}(x, q)) = \text{get}(q)$$

List-based spec:

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

Proof:

Assume $q \neq \text{emptyq}()$

Prove: $\text{get}(\text{enqueue}(x, q)) = \text{last}(\text{cons}(x, q)) = \text{last}(q) = \text{get}(q)$

Implementation

We defined $\mathcal{Q}$ by extending $\mathcal{L}$ (`queue_ext.hs`)

Must implementations of this ADT **use lists**?

$$\text{emptyq}() = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \text{enqueue}(x, q) = \text{cons}(x, q)$$

$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(q)$$

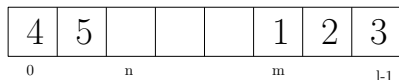
$$\forall q \in \mathcal{Q} . q \neq \langle \rangle \Rightarrow \text{dequeue}(q) = \text{dropLast}(q)$$

We implicitly assumed $\mathcal{Q}$ and $\mathcal{L}$ to be identical.

Now: Drop this assumption. Implementation without lists

Implementation Based on Arrays

Intuition: Represent the list by an array, e.g. $\langle 1, 2, 3, 4, 5 \rangle$ by



More concretely: $\mathcal{Q} := \mathcal{A} \times \mathcal{N} \times \mathcal{N}$, where

(a, m, n) represents $\langle a[m], a[(m+1) \bmod l], \dots, a[n-1] \rangle$, for some constant l .

Formally: $\rho(a, m, n) := \begin{cases} \langle \rangle, & \text{if } m = n, \text{ and} \\ \text{cons}(a[m], \rho(a, (m+1) \bmod l, n)), & \text{otherwise.} \end{cases}$

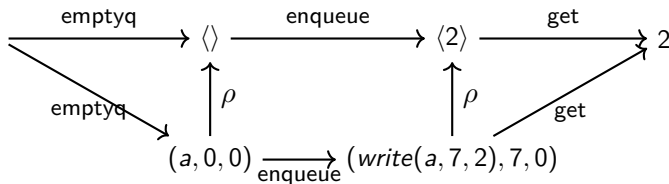
Implementation as above!

(we will use the notation $i^- := (i-1) \bmod l$)

`(queue_arr.hs)`

correct?

Checking Correctness: Example:



Example, according to spec:

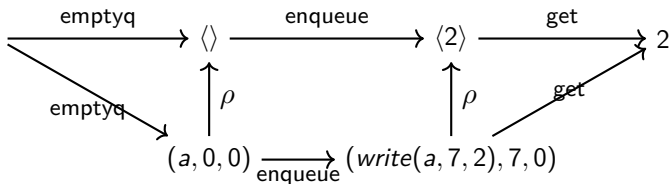
$$\text{get}(\text{enqueue}(2, \text{emptyq}())) = \text{get}(\text{enqueue}(2, \langle \rangle)) = \text{get}(\langle 2 \rangle) = 2$$

Implementation ($I = 8$):

$$\begin{aligned} \text{get}(\text{enqueue}(2, \text{emptyq}())) &= \text{get}(\text{enqueue}(2, (a, 0, 0))) = \\ &\text{get}((\text{write}(a, 7, 2), 7, 0)) = \text{write}(a, 7, 2)[7] = 2 \end{aligned}$$

representation function ρ

Checking Correctness



List-based spec:

$$\rho(\text{emptyq}()) = \langle \rangle$$

$$\forall x \in \mathcal{N}, q \in \mathcal{Q} . \rho(\text{enqueue}(x, q)) = \text{cons}(x, \rho(q))$$

$$\forall q \in \mathcal{Q} . \rho(q) \neq \langle \rangle \Rightarrow \text{get}(q) = \text{last}(\rho(q))$$

$$\forall q \in \mathcal{Q} . \rho(q) \neq \langle \rangle \Rightarrow \rho(\text{dequeue}(q)) = \text{dropLast}(\rho(q))$$

Now: $\mathcal{Q}$: our array based implementation

Checking Correctness

List-based spec:

$$\rho(\text{emptyq}()) = \langle \rangle$$

$$\forall x \in \mathcal{N}, (a, m, n) \in \mathcal{Q} . \rho(\text{enqueue}(x, (a, m, n))) = \text{cons}(x, \rho(a, m, n))$$

$$\forall (a, m, n) \in \mathcal{Q} . \rho(a, m, n) \neq \langle \rangle \Rightarrow \text{get}(a, m, n) = \text{last}(\rho(a, m, n))$$

$$\forall (a, m, n) \in \mathcal{Q} . \rho(a, m, n) \neq \langle \rangle \Rightarrow \rho(\text{dequeue}(a, m, n)) = \text{dropLast}(\rho(a, m, n))$$

Let us check this.

`(queue_arr_ass.hs)`

$$\rho(\text{emptyq}()) = \langle \rangle$$

$$\rho(\text{emptyq}()) = \rho(a, 0, 0) = \langle \rangle$$

$$\rho(\text{enqueue}(x, (a, m, n))) = \text{cons}(x, \rho(a, m, n))$$

By definition,

$$\rho(\text{enqueue}(x, (a, m, n))) = \rho(\text{write}(a, m^-, x), m^-, n).$$

According to the definition of ρ , if $m^- \neq n$, this is

$$\text{cons}(\text{write}(a, m^-, x)[m^-], \rho(a, (m^- + 1) \bmod l, n)),$$

which, due to the array axioms and modular arithmetic is

$$\text{cons}(x, \rho(a, m, n)).$$

If $m^- = n$? $\langle \rangle$?

6	7		1	2	3	4	5
		n	m				

bug (overflow)

$$\rho(a, m, n) := \begin{cases} \langle \rangle, & \text{if } m = n, \text{ and} \\ \text{cons}(a[m], \rho(a, (m + 1) \bmod l, n)), & \text{otherwise.} \end{cases}$$

$$\rho(a, m, n) \neq \langle \rangle \Rightarrow \text{get}(a, m, n) = \text{last}(\rho(a, m, n))$$

We assume $(a, m, n) \neq \langle \rangle$, and prove $\text{get}(a, m, n) = \text{last}(\rho(a, m, n))$

By definition, $\text{get}(a, m, n) = a[n^-]$.

We prove that this is equal to $\text{last}(\rho(a, m, n))$.



If $m = n^-$,

$$\begin{aligned} \text{last}(\rho(a, m, n)) &= \text{last}(\text{cons}(a[m], \rho(a, n, n))) = \\ &= \text{last}(\text{cons}(a[m], \langle \rangle)) = a[m] = a[n^-]. \end{aligned}$$

Now we proceed by induction: We assume

$$\text{last}(\rho(a, m, n)) = a[n^-],$$

and prove

$$\text{last}(\rho(a, m^-, n)) = a[n^-].$$

Induction Step

We assume

$$\text{last}(\rho(a, m, n)) = a[n^-],$$

and prove

$$\text{last}(\rho(a, m^-, n)) = a[n^-].$$

We have:

$$\begin{aligned}\text{last}(\rho(a, m^-, n)) &= \text{last}(\text{cons}(a[m^-], \rho(a, m, n))) = \\ &= \text{last}(\rho(a, m, n)) = a[n^-],\end{aligned}$$

which is, what we wanted to prove.

Wait! $m^- = n$: overflow!

$$\rho(a, m, n) \neq \langle \rangle \Rightarrow \rho(\text{dequeue}(a, m, n)) = \text{dropLast}(\rho(a, m, n))$$

similar

Methods for Specification of Data Types

- ▶ Property based:
 - ▶ axiomatic
 - ▶ algebraic (axioms restricted to equalities)

Example: Alloy (<https://alloytools.org>)

- ▶ Model based: list possible behavior instead of describing properties
Examples:
 - ▶ Z notation
 - ▶ Vienna Development Method (VDM)
 - ▶ TLA+ (Amazon, Microsoft)

Conclusion

OO $\approx$ abstract data types = logical theories

Barbara H Liskov and Jeannette M Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6): 1811–1841, 1994.